COMP1531
Software Engineering Fundamentals

Week 1
Git - Solo Usage

The Problem

To effectively work on **large projects** with **multiple engineers,** we need a more advanced way to manage our codebase.

- Simple file sharing is not enough.

The Problem

Why Simple File Sharing Fails?

Tools like:

Dropbox ??OneDrive ?? Google Docs ??

Provides:

- File syncing
- Basic version history
- Collaboration features

However, they are:

❌ Too simple for complex software projects

❌ Not designed specifically for programming

❌ Limited in handling large-scale collaboration

The Problem

What We Actually Need

A more powerful system that supports:

1. **Version Control**
   - Tracks changes to our code over time in a detailed and systematic way (like a logbook and/or time machine)

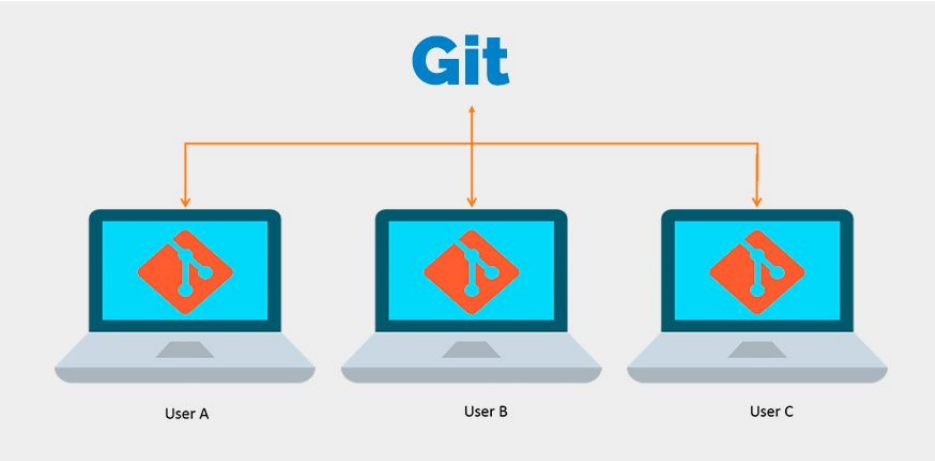
2. **Concurrent Programming (Collaboration)**
   - Effectively allows multiple people to work on the same files or series of files and seamlessly integrate changes together (like google docs but for code).

A Popular Solution: Git

- Built for programmers to work concurrently on the same project
- Designed for managing code
  - Across lots of people
  - Detailed history

What Solution does Git provide?

- Distributed version control
- Each user has a full backup
- Work is also in the cloud



source

Git? GitHub? GitLab? Bitbucket?

- Git is just a command line program
  - it tracks changes in files and helps teams build software together safely and efficiently.
- GitHub, GitLab, and Bitbucket are all software tools that implement the git language via an “easy to use” web app

- In this course, we will use GitLab

Learning Git

We're going to use git in 3 ways:

1. Version control on a single machine
2. Version control across multiple of your machines
3. Version control across a team of engineers

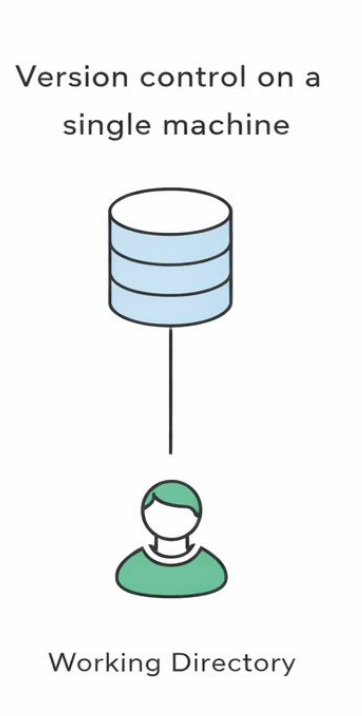
These will be practical demos. If you want to follow a written guide, then please checkout Atlassian's git guide.

Learning Git

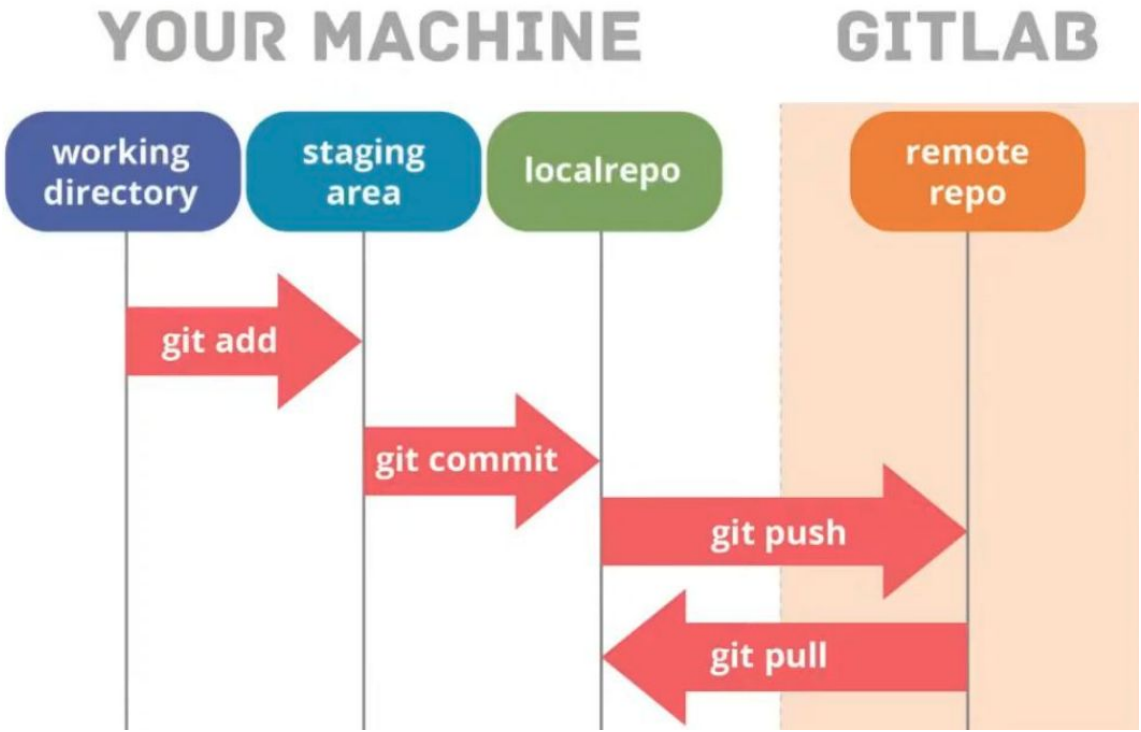
Single Machine

Stage 1: Version control on a single machine

setup	status of work	doing work
● SSH Keys ● git clone	● git status ● git log	● git add ● git diff ● git commit ● git push



Git Operations - Single Machine

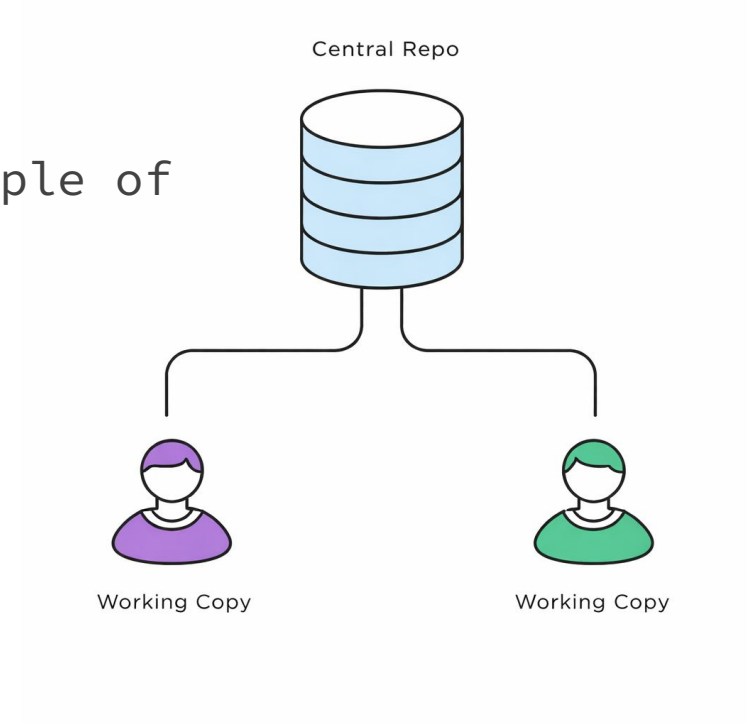


Learning Git

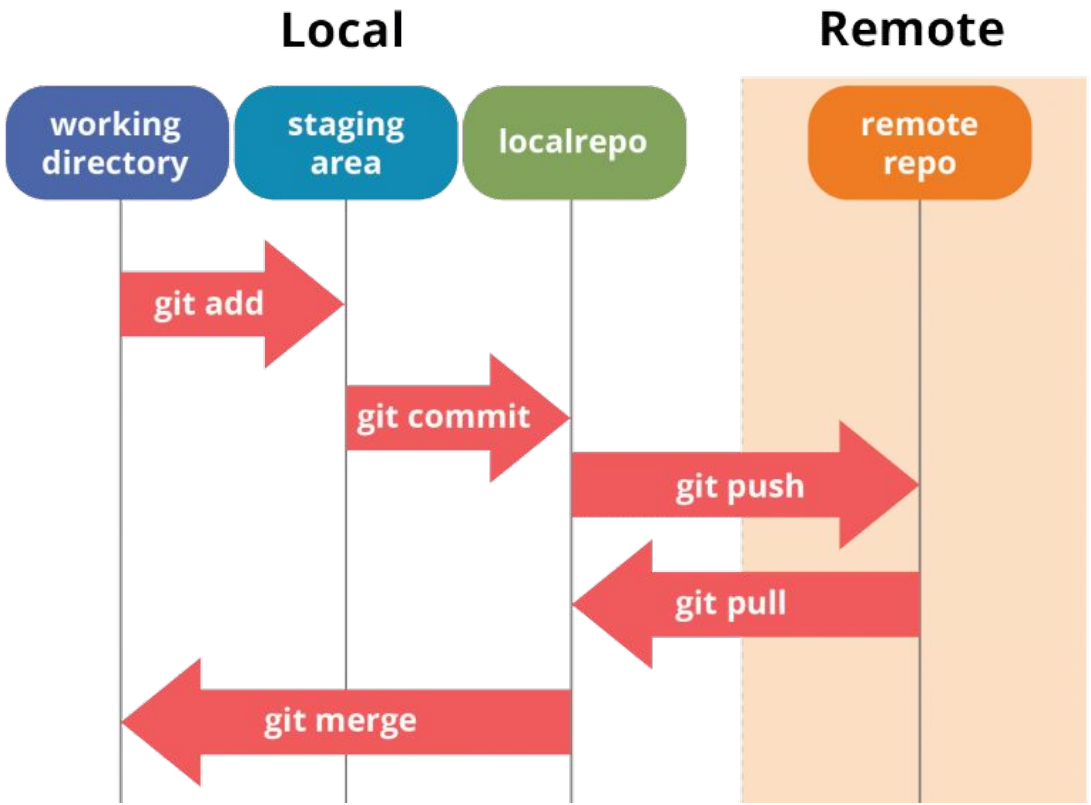
Multiple Of Your Machines

Stage 2: Version control across multiple of your machines

multiple machines
- git pull
- merge conflicts



Git Operations - Multiple Of Your Machines



Git Commands Summary

Command	Description	Example
`git clone`	Clones a repository from a cloud/remote repository to a local repository	`git clone <repo-url>`
`git status`	Shows the current state of your repository (staged, unstaged, untracked files)	`git status`
`git log`	Displays the commit history	`git log`
`git add`	Stages changes for commit (adds untracked files or stages modified files)	`git add --all` `git add file.py`
`git diff`	Shows differences between the last commit and current changes	`git diff`
`git commit`	Commits staged changes (takes a snapshot of your work)	`git commit -m "Message name"`
`git push`	Syncs local commits to the remote/cloud repository	`git push` `git push origin master`
`git pull`	Syncs changes from the remote/cloud repository to your local repository	`git pull` `git pull origin master`

Feedback

Or go to the form here